Week 1 - Skill Builder
****

1. Sita wants to count the number of fruits she has: apples, bananas, and cherries. Declare variables to store these quantities and print each with an appropriate message.

apples (integer): stores number of apples, 10
bananas (integer): stores number of bananas, 20
cherries (integer): stores number of cherries, 30

Input format :
No console input.

Output format :
The first line of output prints: "Number of apples is <apples>"
The second line of output prints: "Number of bananas is <bananas>"
The third line of output prints: "Number of cherries is <cherries>"
Refer to the sample output for formatting specifications.

Sample test cases :
Input 1 :
Output 1 :
Number of apples is 10
Number of bananas is 20
Number of cherries is 30

Output
****

```
apple = 10
banana = 20
cherries = 30
print(f"Number of apples is {apple}")
print(f"Number of bananas is {banana}")
print(f"Number of cherries is {cherries}")
```

2. Alice wants to share her personal details: name, age, and city of residence. Declare variables to store this information and print each detail on a new line with a descriptive message.
name (string): stores Alice’s name, "Alice"

age (integer): stores Alice's age, 25

city (string): stores Alice's city, "Delhi"

Input format :
No console input.

Output format :
The first line of output prints: "Name is <name>"
The second line of output prints: "Age is <age>"
The third line of output prints: "City is <city>"
Refer to the sample output for formatting specifications.

Sample test cases :
Input 1 :
Output 1 :
Name is Alice
Age is 25
City is Delhi

Output
****

```
name = "Alice"
age = 25
city = "Delhi"
print(f"Name is {name}")
print(f"Age is {age}")
print(f"City is {city}")
```

3.John's user profile tracks three boolean statuses: whether the user is active, an admin, and verified. Declare boolean variables to store these statuses and print each status with a descriptive message.

is_active (boolean): stores if the user is active (True)
is_admin (boolean): stores if the user is an admin (False)
is_verified (boolean): stores if the user is verified (True)

Input format :
No console input.

Output format :
The first line of output prints: "User active status is <is_active>"
The second line of output prints: "User admin status is <is_admin>"
The third line of output prints: "User verified status is <is_verified>"

Sample test cases :
Input 1 :
Output 1 :
User active status is True
User admin status is False
User verified status is True

Output
****
is_active = True
is_admin = False
is_verified = True
print(f"User active status is {is_active}")
print(f"User admin status is {is_admin}")
print(f"User verified status is {is_verified}")

4. Fazil needs a triangle classification program. Given the lengths of the sides of a triangle (side1, side2, and side3), the program determines whether the triangle is classified as Equilateral, Isosceles, or Scalene based on the following conditions:

If all sides are equal, classify the triangle as Equilateral.
If exactly two sides are equal, classify the triangle as Isosceles.
If all sides are different, classify the triangle as Scalene.

Input format :
The first line of input consists of a float value A, representing the length of the first side of the triangle.
The second line of input consists of a float value B, representing the length of the second side of the triangle.
The third line of input consists of a float value C, representing the length of the third side of the triangle.

Output format :
The output displays "The triangle is classified as X" where X is the string representing the classification of the triangle as "Equilateral", "Isosceles", or "Scalene".

Sample test cases :
Input 1 :
90.0
90.0
90.0

Output 1 :

The triangle is classified as Equilateral

Input 2 :
4.5
4.5
6.8

Output 2 :
The triangle is classified as Isosceles

Input 3 :
12.3
15.7
17.8

Output 3 :
The triangle is classified as Scalene

Output
****

```
A = float(input())
B = float(input())
C = float(input())

if A==B and B==C and C==A:
    print("The triangle is classified as Equilateral")
elif A==B or B==C:
    print("The triangle is classified as Isosceles")
else:
    print("The triangle is classified as Scalene")
```

5. Banu is an astrologer who specializes in identifying people's zodiac signs based on their birthdates. She wants to develop a simple tool that can assist her in determining the zodiac signs of individuals. She's seeking your help to write a program that automates this process.
Zodiac Sign Determination:

Aries: March 21–April 19
Taurus: April 20–May 20
Gemini: May 21–June 20
Cancer: June 21–July 22
Leo: July 23–August 22
Other: For dates outside of the above ranges

Input format :
The first line consists of an integer representing the day of birth (1–31).
The second line consists of a string representing the month of birth.

Output format :
The output displays "Your zodiac sign is " followed by the zodiac sign corresponding to the given birthdate.

Sample test cases :
Input 1 :
15
April

Output 1 :
Your zodiac sign is Aries

Input 2 :
2
May

Output 2 :
Your zodiac sign is Taurus

Input 3 :
5
June

Output 3 :
Your zodiac sign is Gemini

Input 4 :
21
June

Output 4 :
Your zodiac sign is Cancer

Input 5 :
30
July

Output 5 :
Your zodiac sign is Leo

Input 6 :
23
September

Output 6 :
Your zodiac sign is Other

Output
****

```
date = int(input())
month = input()
print("Your zodiac sign is",end=" ")

if date >=21 and month == "March" or date<=19 and month=="April":
    print("Aries")
elif date >=20 and month == "April" or date<=20 and month=="May":
    print("Taurus")
elif date >=21 and month == "May" or date<=20 and month=="June":
    print("Gemini")
elif date >=21 and month == "June" or date<=22 and month=="July":
    print("Cancer")
elif date >=23 and month == "July" or date<=22 and month=="August":
    print("Leo")
else:
    print("Other")
```

Week 2 - Skill Builder
****

1. Alice works at a digital marketing company, where she analyzes large datasets. One day, she's tasked with processing customer ID numbers, which are long numeric sequences.
To simplify her task, Alice needs to calculate the digital root of each ID. The digital root is obtained by repeatedly summing the digits of a number until a single digit remains.
Help Alice write a program that reads a customer ID number, calculates its digital root, and prints the result using a loop-based approach.
For example, the sum of the digits of 98675 is 9 + 8 + 6 + 7 + 5 = 35, then 3 + 5 = 8, which is the digital root.

Function prototype: def digital_root(num)

Input format :
The input consists of an integer num.

Output format :
The output prints an integer representing the sum of digits for a given number until a single digit is obtained.

Sample test cases :
Input 1 :
451110

Output 1 :
3

Input 2 :
1234

Output 2 :
1

Input 3 :
98675

Output 3 :
8

Output
****

```
def digital_root(num):
    sum = 0
    while num:
        last = num % 10
        sum += last
        num //= 10
        if sum > 9 and num == 0:
            num = sum
            sum = 0
    return sum
```

2. Sophia is developing a feature for her online banking application that calculates the total sum of digits in customers' account numbers. This sum is used to generate unique verification codes for secure transactions. She needs a program that takes an account number as input and outputs the sum of its digits.

Help Sophia to complete her task.

Function Specification: def sum_digits(num)

Input format :
The input consists of an integer, representing the customer's account number.

Output format :
The output prints an integer representing the sum of the digits of the account number.

Sample test cases :
Input 1 :
123245

Output 1 :
17

Input 2 :
0

Output 2 :
0

Output
****
```
def sum_digits(num):
    sum = 0
    while num:
        last = num % 10
        sum += last
        num //= 10
    return sum
```

3. Sophia manages the payroll at a local business where employees are paid based on the monthly hours they work. She needs to calculate the monthly salary for each employee, considering a fixed hourly rate and standard deductions. The deductions amount to 15% of the gross salary.
Help Sophia write a program that calculates each employee's net monthly salary. The program should read the total hours worked and the hourly rate from the input, compute the gross salary, apply the deductions, and then output the net salary.

Example

Input

40

12.50

Output

Hours Worked: 40.0 hours

Hourly Rate: 12.50 per hour

Net Salary: 425.00

Explanation

The gross salary is calculated as 40.0 hours * 12.50/hour = 500.0

The deductions are 15% of the gross salary, which is 0.15 * 500.00 = 75.00

The net salary is gross - deductions = 500.0 - 75.0 = 425.00.

Input format :

The first line contains a positive floating-point number representing the number of hours worked by the employee.

The second line contains a positive floating-point number representing the hourly rate for the employee.

Output format :

The first line displays the number of hours worked in the format "Hours Worked: {hours} hours," where {hours} is the actual number of hours worked with two decimal places.

The second line displays the hourly rate in the format "Hourly Rate: {rate} per hour," where {rate} is the actual hourly rate rounded to two decimal places.

The third line displays the net salary in the format "Net Salary: {salary}," where {salary} is the actual net salary rounded to two decimal places.

Sample test cases :

Input 1 :

40.00

12.50

Output 1 :

Hours Worked: 40.00 hours

Hourly Rate: 12.50 per hour

Net Salary: 425.00

Input 2 :
80.00
30.75

Output 2 :
Hours Worked: 80.00 hours
Hourly Rate: 30.75 per hour
Net Salary: 2091.00

Output
****

```
def calculate_gross_salary(hours_worked, hourly_rate):
    return hours_worked * hourly_rate

def calculate_deductions(gross_salary):
    return gross_salary * 0.15

def calculate_net_salary(gross_salary, deductions):
    return gross_salary - deductions

def print_salary_details(hours_worked, hourly_rate, monthly_salary):
    print(f"Hours Worked: {hours_worked:.2f} hours")
    print(f"Hourly Rate: {hourly_rate:.2f} per hour")
    print(f"Net Salary: {monthly_salary:.2f}")
```

4. Sarah works at a retail store and needs a program to help customers quickly calculate the total price of an item after adding sales tax. The store applies a fixed sales tax rate of 8% (0.08), defined as a global constant.
Write a function that takes the item's cost as input, computes the sales tax, and returns the total cost. The program should then display the original cost, the tax rate, and the final total price to the customer. Help Sarah provide accurate pricing with this simple calculator!

Function Signature: total_cost(item_cost)

Input format :
The input consists of a single line containing a positive floating-point number representing the cost of the item.

Output format :
The first line of output prints "Item Cost: " followed by the item's cost formatted to two decimal places.

The second line prints "Sales Tax Rate: " followed by the sales tax rate as a percentage with a % symbol.
The third line prints "Total Cost: " followed by the total cost after applying sales tax, formatted to two decimal places.

Sample test cases :
Input 1 :
50.00

Output 1 :
Item Cost: 50.00
Sales Tax Rate: 8.0%
Total Cost: 54.00

Input 2 :
100.5

Output 2 :
Item Cost: 100.50
Sales Tax Rate: 8.0%
Total Cost: 108.54

Output
****

```
def total_cost(item_cost):
    total = item_cost + (item_cost * 0.08)
    print(f"Item Cost: {item_cost:.2f}")
    print("Sales Tax Rate: 8.0%")
    print(f"Total Cost: {total:.2f}")


item_cost = float(input())
total_cost(item_cost)
```

5. Arjun is building a text analyzer for his English assignment. He needs to create a function that can count how many vowels are present in a given string. The string contains both lowercase and uppercase letters.
Help him implement this function that takes an input string and returns the count of vowels in the string.

Function Signature: count_vowels(input_string)

Input format :
The input consists of a single line containing a string.

Output format :
The output consists of a single line containing an integer representing the number of vowels (both lowercase and uppercase) found in the input string in the format:
"Number of vowels in the string: [number of vowels]"

Sample test cases :
Input 1 :
Hello, World!

Output 1 :
Number of vowels in the string: 3

Input 2 :
Python is a great programming language.

Output 2 :
Number of vowels in the string: 12

Input 3 :
APPle

Output 3 :
Number of vowels in the string: 2

Output
****

```
def count_vowels(string):
    vowels = "aeiouAEIOU"
    count = 0
    for i in string:
        if i in vowels:
            count +=1
    print(f"Number of vowels in the string: {count}")
string = input()
count_vowels(string)
```

—----------

Week 3 - Skill Builder
****

1. Sarah is a technical writer who is responsible for formatting two important documents. Both documents contain a certain placeholder character that needs to be replaced with another character before they can be finalized. To ensure consistency in formatting, Sarah wants you to help her write a program that processes both documents by replacing the placeholder character with the new one. Sarah also prefers a neat and structured output, so she wants you to ensure that both modified documents are printed in a single line, separated by a space, using the format() function.

<u>Example</u>

Input:

Hello

World

o

a

Output:

Hella Warld

Explanation:

Here the character 'o' is replaced with 'a' in the concatenated string.

Input format :
The first line contains string1, the first document.
The second line contains string2, the second document.
The third line contains char1, the placeholder character that needs to be replaced.
The fourth line contains char2, the new character that will replace the placeholder.

Output format :
The output displays a single line containing the modified string1 and string2, separated by a space.

Sample test cases :
Input 1 :
Hello
World
o

a

Output 1 :
Hella Warld

Input 2 :
12345
45123
1
0

Output 2 :
02345 45023

Input 3 :
python programming
Practice
p
P

Output 3 :
Python Programming Practice

Output
****

```
str1 = input()
str2 = input()

R = input()
N = input()

str1 = str1.replace(R,N)
str2 = str2.replace(R,N)
print(str1,str2)
```

2. John Smith is a new employee at the company "iamNeo" and has been assigned the role of an admin. To create an email address for him, the system uses a naming convention where the first letter of his first name is followed by his last name, with the suffix based on his role. Admins get a specific email domain format (@admin.company.com), while other roles use a generic domain (@company.com). Help him generate his email address using this format.

Input format :

The first line of input consists of a string first_name, representing the first name of the employee.
The second line consists of a string last_name, representing the last name of the employee.
The third line consists of a string role, representing the role of the employee.
The fourth line consists of a string company, representing the company name.

Output format :
The output prints the generated email address, followed by a string representing the complete email address using the format first letter of first name + last name + domain.

Sample test cases :
Input 1 :
John
Smith
admin
iamNeo

Output 1 :
jsmith@admin.iamneo.com

Input 2 :
Anne
Cuthbert
IT
iamNeo

Output 2 :
acuthbert@iamneo.com

Output
****

```
firstname = input()
lastname = input()
role = input()
company = input()

name = firstname[0].lower()+lastname.lower()
if role == "admin":
    print(f"{name}@{role}.{company}.com")
else:
    print(f"{name}@{company}.com")
```

3. Rekha is working on a project where she needs to combine two strings by alternating their characters. She has two strings str1 and str2, both of equal length. She wants to create a new string where the first character of str1 is followed by the first character of str2, then the second character of str1 followed by the second character of str2, and so on.
Help her to implement the task.

Input format :
The first line of input consists of a string str1.
The second line of input consists of a string str2.

Output format :
The output displays the concatenated string in the mentioned format.

Sample test cases :
Input 1 :
abc
def

Output 1 :
adbecf

Input 2 :
140401

Output 2 :
1-4-0-4-0-1-

Input 3 :
test case
case test

Output 3 :
tceasste ctaesset

Output
****

```
str1 = input()
str2 = input()
for i in range(len(str1)):
    print(str1[i]+str2[i],end="")
```

4. Ram is working on a program to manipulate strings. He wants to create a program that takes two strings as input, reverses the second string, and then concatenates it with the first string.
Ram needs your help to design a program.

Input format :
The first line of input contains a string representing the first string.
The second line of input contains a string representing the second string.

Output format :
The output displays a string representing the concatenated string of the first string and the reversed second string.

Sample test cases :
Input 1 :
hello
word

Output 1 :
hellodrow

Input 2 :
ram@123
mailid

Output 2 :
ram@123diliam

Input 3 :
the_sun
moon

Output 3 :
the_sunnoom

Output
****

```
str1 = input()
str2 = input()

str2 = str2[::-1]
print(str1+str2)
```

5. Ravi is designing a contact list parser that processes multiple phone numbers stored as strings. Each phone number follows the format "(XXX) XXX-XXXX". He aims to extract all valid area codes from the given list and output them in a comma-separated string.
A phone number is considered valid only if:

- It follows the format (XXX) XXX-XXXX
- All characters in X are digits only
- The total length is exactly 14 characters, including parentheses, spaces, and hyphen

You need to help Ravi write a Python program that reads multiple such strings, validates them, extracts the area codes of valid numbers, and prints them in a comma-separated format.

Input format :
The first line contains an integer n representing the number of phone numbers.
The next n lines each contain a string, where each string represents a phone number.

Output format :
The output displays a single line: "Valid area codes: " followed by the comma-separated area codes extracted from valid phone numbers only.

Sample test cases :
Input 1 :
3
(123) 456-7890
(abc) 111-2222
(987) 654-3210

Output 1 :
Valid area codes: 123,987

Input 2 :
3
(000) 000-0000
(999) 999-9999
(123 456-7890

Output 2 :
Valid area codes: 000,999

Input 3 :
5

(789) 123-4567
456 789-1234
(999 888-7777
(000)000-0000
(321) 654-0987

Output 3 :
Valid area codes: 789,321

Output
****

```
n = int(input())
valid = ""
for i in range(n):
    s = input()
    if s[0] == "(" and s[4] == ")" and s[1:4].isdigit() and s[6:9].isdigit() and s[-4:].isdigit():
        valid +=s[1:4] +","

print(f"Valid area codes: {valid.rstrip(',')}")
```

—----------

Week 4 - Skill Builder
****

1. Tom wants to create a dictionary that lists the first n prime numbers, where each key represents the position of the prime number, and the value is the prime number itself.
Help Tom generate this dictionary based on the input she provides.

Input format :
The input consists of an integer n, representing the number of prime numbers Tom wants to generate.

Output format :
The output displays the generated dictionary where each key is an integer from 1 to n, and the corresponding value is the prime number.

Sample test cases :
Input 1 :
4

Output 1 :
{1: 2, 2: 3, 3: 5, 4: 7}

Input 2 :
20

Output 2 :
{1: 2, 2: 3, 3: 5, 4: 7, 5: 11, 6: 13, 7: 17, 8: 19, 9: 23, 10: 29, 11: 31, 12: 37, 13: 41, 14: 43, 15: 47, 16: 53, 17: 59, 18: 61, 19: 67, 20: 71}

Output
****

```
n = int(input())
d = {}

num = 2
count = 1

while count <= n:
    prime = True
    for i in range(2,num):
        if num % i == 0:
            prime = False
            break

    if prime:
        d[count] = num
        count +=1

    num += 1
print(d)
```

2. Arjun is a financial analyst at a multinational company. He manages quarterly expense reports submitted by two departments — Marketing and Operations. Each department provides its report as a dictionary, where keys represent expense categories (like “travel”, “supplies”, “advertising”) and values represent total expenses in dollars.
Arjun needs to combine both reports into a single consolidated expense summary. Write a program to add the values for common categories from both dictionaries while retaining unique categories as they are.

Input format :
The first line of input consists of an integer n, representing the number of key-value pairs to be entered for the first dictionary (Marketing).

The next 2n lines contain the key-value pairs for the first dictionary:
Each odd-numbered line (in this block) contains a category (string).
Each even-numbered line contains its corresponding expense amount (integer).
The following line contains an integer m, representing the number of key-value pairs for the second dictionary (Operations).
The next 2m lines contain key-value pairs for the second dictionary in the same format.

Output format :
The output prints the final combined dictionary in the format:
Consolidated Expenses: {key1: value1, key2: value2, ..., keyN: valueN}
For common categories, the values are added together:
combined_value = marketing[key] + operations[key]
For unique categories, the key-value pair is retained as-is from its respective dictionary.
The keys appear in the order they are first encountered across both dictionaries.

Sample test cases :
Input 1 :
2
hardware
1000
maintenance
700
3
maintenance
300
stationery
200
travel
500

Output 1 :
Combined Dictionary: {'hardware': 1000, 'maintenance': 1000, 'stationery': 200, 'travel': 500}

Input 2 :
3
rent
5000
electricity
1200
internet
800

2
internet
400
supplies
600

Output 2 :
Combined Dictionary: {'rent': 5000, 'electricity': 1200, 'internet': 1200, 'supplies': 600}

Output
****

```
n = int(input())
d = {}
for i in range(n):
    key = input()
    value = int(input())
    d[key] = value

m = int(input())
for i in range(m):
    key = input()
    value = int(input())
    if key in d:
        d[key] += value
    else:
        d[key] = value

print(f"Combined Dictionary: {d}")
```

3. Alex is managing an online store and needs to analyze customer orders. He needs to track the order IDs and their corresponding amounts. However, he wants to identify the smallest and largest order amounts based on the order IDs to better understand customer behavior.
Can you help Alex by implementing a program to find the smallest and largest order IDs based on the order amounts?

Input format :
The first line of input contains an integer n, representing the number of orders.
The next 2n lines contain the order data:
The first line contains the order ID (an integer).
The second line contains the order amount (an integer).
Repeat this pattern n times for n orders.

Output format :
The first line of output prints the smallest order ID (based on the minimum order amount), using the format:
Smallest Order ID: {smallest_key},
The second line prints the largest order ID (based on the maximum order amount), using the format:
Largest Order ID: {largest_key}

Sample test cases :
Input 1 :
5
101
200
102
3000
103
100
104
20
105
500

Output 1 :
Smallest Order ID: 104,
Largest Order ID: 102

Input 2 :
4
201
1000
202
500
203
1500
204
800

Output 2 :
Smallest Order ID: 202,
Largest Order ID: 203

Output
****
n = int(input())

```
min_id = max_id = 0
min_amount = float("inf")
max_amount = float("-inf")

for i in range(n):
    id = int(input())
    amount = int(input())

    if amount < min_amount:
        min_amount = amount
        min_id = id

    if amount > max_amount:
        max_amount = amount
        max_id = id

print(f"Smallest Order ID: {min_id},")
print(f"Largest Order ID: {max_id}")
```

4. At a warehouse, two delivery trucks bring boxes of goods every day. Each truck reports the number of boxes delivered to each section of the warehouse.
Write a program that takes two lists of integers as input, representing the number of boxes delivered to each section by each truck, computes the element-wise sum, and outputs a tuple containing the total number of boxes in each section.
This helps the warehouse manager know the combined stock in each section after both deliveries.

Input format :
The first line contains a single integer n, representing the number of sections in the warehouse.
The second line contains n integers separated by commas, representing the boxes delivered by the first truck.
The third line contains n integers separated by commas, representing the boxes delivered by the second truck.

Output format :
The output is a single line containing a tuple of integers, representing the total boxes in each section after combining deliveries.

Sample test cases :
Input 1 :

5
3, 8, 12, 4, 6
2, 7, 5, 6, 9

Output 1 :
(5, 15, 17, 10, 15)

Input 2 :
3
0, 20, 10
5, 15, 25

Output 2 :
(5, 35, 35)

Output
****

```
n = int(input())
delivery_box1 = tuple(int(i) for i in input().split(","))
delivery_box2 = tuple(int(i) for i in input().split(","))
tuple = ()
for i in range(n):
    tuple += (delivery_box1[i] + delivery_box2[i],)

print(tuple)
```

5. Teju is exploring the fascinating world of tuples and their manipulation. She is given a task to transform a list of integers into a tuple and concatenate its elements into a single string.
Help Teju by writing a program that accomplishes this task efficiently.

<u>Example</u>

Input

3 // number of elements in the tuple

1

20

3

Output

(1, 20, 3)

1203

Input format :
The first line of input consists of a single integer n, the number of elements in the tuple.
The next n lines consist of an integer representing the elements in the tuple.

Output format :
The first line of output consists of a tuple containing the elements of the input list in the format: (element1, element2, ...)
The second line of output consists of a single string that represents the concatenation of all elements of the tuple.

Sample test cases :
Input 1 :
3
1
20
3

Output 1 :
(1, 20, 3)
1203

Input 2 :
5
10
20
40
5
70

Output 2 :
(10, 20, 40, 5, 70)
102040570

Output
****

```
n = int(input())
tuple = ()
s = ""
for i in range(n):
    val = int(input())
    tuple += val,
    s += str(val)
```

```
print(tuple)
print(s)
```

—----------

Week 5 - Skill Builder
****

1. A health organization is conducting a Health Check-Up Camp where people undergo various tests, including Body Mass Index (BMI) calculation.
As part of this initiative, you are required to develop a program that records participants' details and calculates their BMI.

The class should have the following methods:

bmi(): calculates the body mass index (BMI) of the person using the formula BMI = weight / height2.
str(): returns a string representation of the person

Create a list of Person objects with different attribute values. Then, iterate through the list and print out the string representation of each person.

Input format :
The first line of input consists of the number of people.
For each person, the input consists of the name as a string, age as an integer, gender as a string, height (in meters) as a float value, and weight (in kg) as a float value in each line.

Output format :
The output prints the string representation of the person in the following format:
"Name: <name>, Age: <age>, Gender: <gender>, Height: <height> m, Weight: <weight> kg, BMI: <bmi_value>"

Sample test cases :
Input 1 :

```
2
Alice
30
Female
1.8
75
Danny
32
Male
2.8
78
```

Output 1 :
Name: Alice, Age: 30, Gender: Female, Height: 1.8 m, Weight: 75.0 kg, BMI: 23.15
Name: Danny, Age: 32, Gender: Male, Height: 2.8 m, Weight: 78.0 kg, BMI: 9.95

Input 2 :
2
Michael
40
Male
1.8
90
Sarah
28
Female
1.6
65

Output 2 :
Name: Michael, Age: 40, Gender: Male, Height: 1.8 m, Weight: 90.0 kg, BMI: 27.78
Name: Sarah, Age: 28, Gender: Female, Height: 1.6 m, Weight: 65.0 kg, BMI: 25.39

Output
****

```
class Person:
    def __init__(self,n,a,g,h,w):
        self.n = n
        self.a = a
        self.g = g
        self.h = h
        self.w = w

    def bmi(self):
        return round(self.w/self.h**2,2)

    def __str__(self):
        return f"Name: {self.n}, Age: {self.a}, Gender: {self.g}, Height: {self.h} m, Weight: {self.w} kg, BMI: {self.bmi()}"
```

2. A banking system is introducing a new feature that allows customers to store and manage money in rupees and paisa. Customers frequently deal with transactions that involve paisa, and sometimes, when the paisa sum exceeds 100, it needs to be converted into rupees.

As a developer, you need to write a program that helps customers add two money amounts correctly, ensuring that any extra paisa is converted into rupees.
Implement a Python program using a class and objects.

Input format :
The first line contains two integers representing rupees and paisa for the first amount (separated by spaces).
The second line contains two integers representing rupees and paisa for the second amount (separated by spaces).

Output format :
The output prints the total sum of the two amounts, rounding off to two decimal places.

Sample test cases :
Input 1 :
50 85
42 65

Output 1 :
93.50

Input 2 :
254 45
845 65

Output 2 :
1100.10

Input 3 :
54 25
35 75

Output 3 :
90.00

Output
****

```
class Money:
    def __init__(self,r,p):
        self.r = r
        self.p = p
```

```
    def add(self,other):
        self.rupees = self.r + other.r
        self.paisa = self.p + other.p

        if self.paisa >= 100:
            self.rupees += self.paisa // 100
            self.paisa %=100

        return f"{self.rupees}.{self.paisa:02d}"
```

3. A digital security firm is analyzing numeric codes to strengthen their authentication system. Each time a user enters a numeric code, the system must determine how many digits the code contains to validate its length.
As a developer, you need to build a program that counts the number of digits in a given number.
Implement a Python program using a class named DigitsOpr with the following specifications:
Methods:

getNum(self): Accepts a positive integer input from the user.
countDigits(self): Counts the number of digits in the given input and returns the count.

Input format :
The input consists of a positive integer N.

Output format :
The output prints an integer representing the total number of digits in the given input.

Sample test cases :

Input 1 :
12345

Output 1 :
5

Input 2 :
2

Output 2 :
1

Output
****

```
class DigitsOpr:
    count = 0
    def getNum(self,n):
        while n:
            n//=10
            self.count+=1

    def countDigits(self):
        return self.count
```

4. A language learning app is developing a new feature to help users understand vowels and consonants better. As part of this feature, users can enter a single alphabet, and the system will classify it as either a vowel or a consonant.
As a developer, your task is to create a program that checks whether a given character is a vowel or a consonant.

Implement a Python program using a class named CharacterChecker with the following specifications:

Methods:

checkVowelOrConsonant(self, char): Checks if the given character is a vowel or a consonant and returns the result.

Input format :
The input consists of a single alphabet character (uppercase or lowercase).

Output format :
The output prints "The character <char> is a consonant" or "The character <char> is a vowel".

Sample test cases :
Input 1 :
j

Output 1 :
The character 'j' is a consonant

Input 2 :
E

Output 2 :
The character 'E' is a vowel

Output

****

```
class CharacterChecker:
    def setCharacter(self,c):
        self.c = c

    def checkVowelOrConsonant(self):
        if self.c.lower() in "aeiou":
            return f"The character '{self.c}' is a vowel"
        else:
            return f"The character '{self.c}' is a consonant"
```

—----------